Welcome to SEAS Online at George Washington University

Class will begin shortly

Audio: To eliminate background noise, please be sure your audio is muted. To speak, please click the hand icon at the bottom of your screen (**Raise Hand**). When instructor calls on you, click microphone icon to unmute. When you've finished speaking, ***be sure to mute yourself again***.

Chat: Please type your questions in Chat.

Recordings: As part of the educational support for students, we provide downloadable recordings of each class session to be used exclusively by registered students in that particular class. Releasing these recordings is strictly prohibited.

THE GEORGE WASHINGTON UNIVERSITY
WASHINGTON, DC

SEAS 8515 – Data Engineering for AI Lecture 1

Walid Hassan, M.B.A, D.Eng.

Agenda

- Introductions
- Syllabus Walk-Thru
- Lecture 1: Spark Environment Setup

Ice Breaker

Please introduce yourself:

- Name
- Location
- Education: Undergrad/Grad
- Profession:
  - Any background in programming (scale 1-5)? Python?Spark? Machine (Windows/Mac)?
- What do you hope to get out of this class?
- Hobbies/Interests (and anything else)

Ice Breaker

Please introduce yourself:

- Name: Walid Hassan (Waleed)
- Location: Dallas, Texas
- Education: BE, M.B.A, D.Eng.
- Profession: Senior IT Executive

Syllabus

Syllabus
Walk Thru

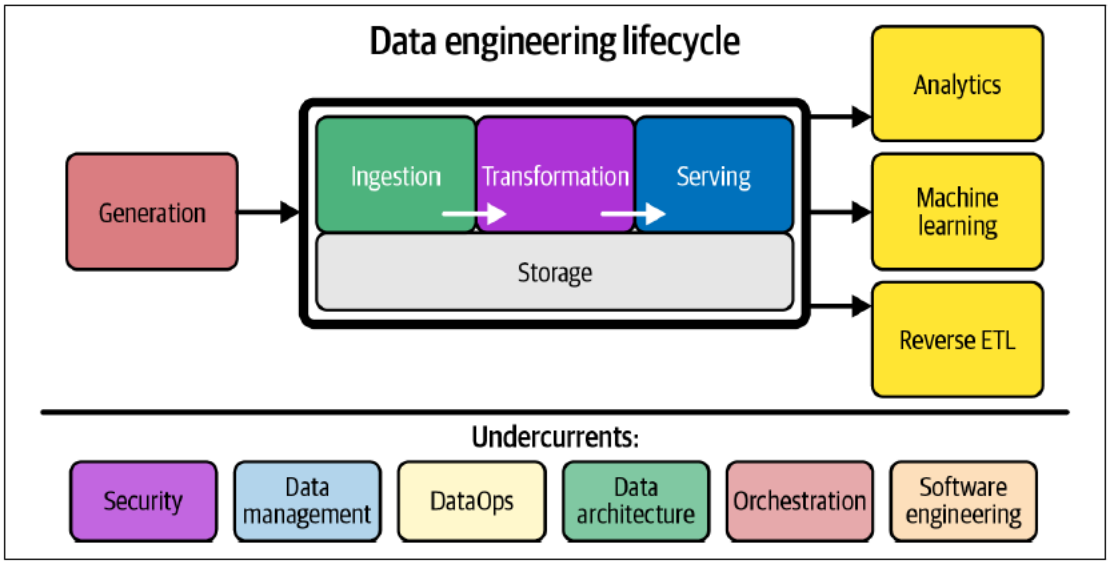
Syllabus

- Please email me at wnhassan@gwu.edu (do not send emails thru blackboard) and include SEAS8515 in email subject.
- Class attendance is automatically captured and reported (prior notice is required for not being able to attend with valid reason).
- The lecture notes/discussions/assignments are in scope for exams.

Syllabus

- Assignments should be submitted on time (due 9 AM EST morning of next class). Late submissions are not accepted.
- All submissions are individual work deliverables.
- Assignments should be submitted in the following format:
  - HW_Number_LastName_FirstName
  - Please make sure you answer all the parts of the assignments as per the instructions.

Data Engineering Defined



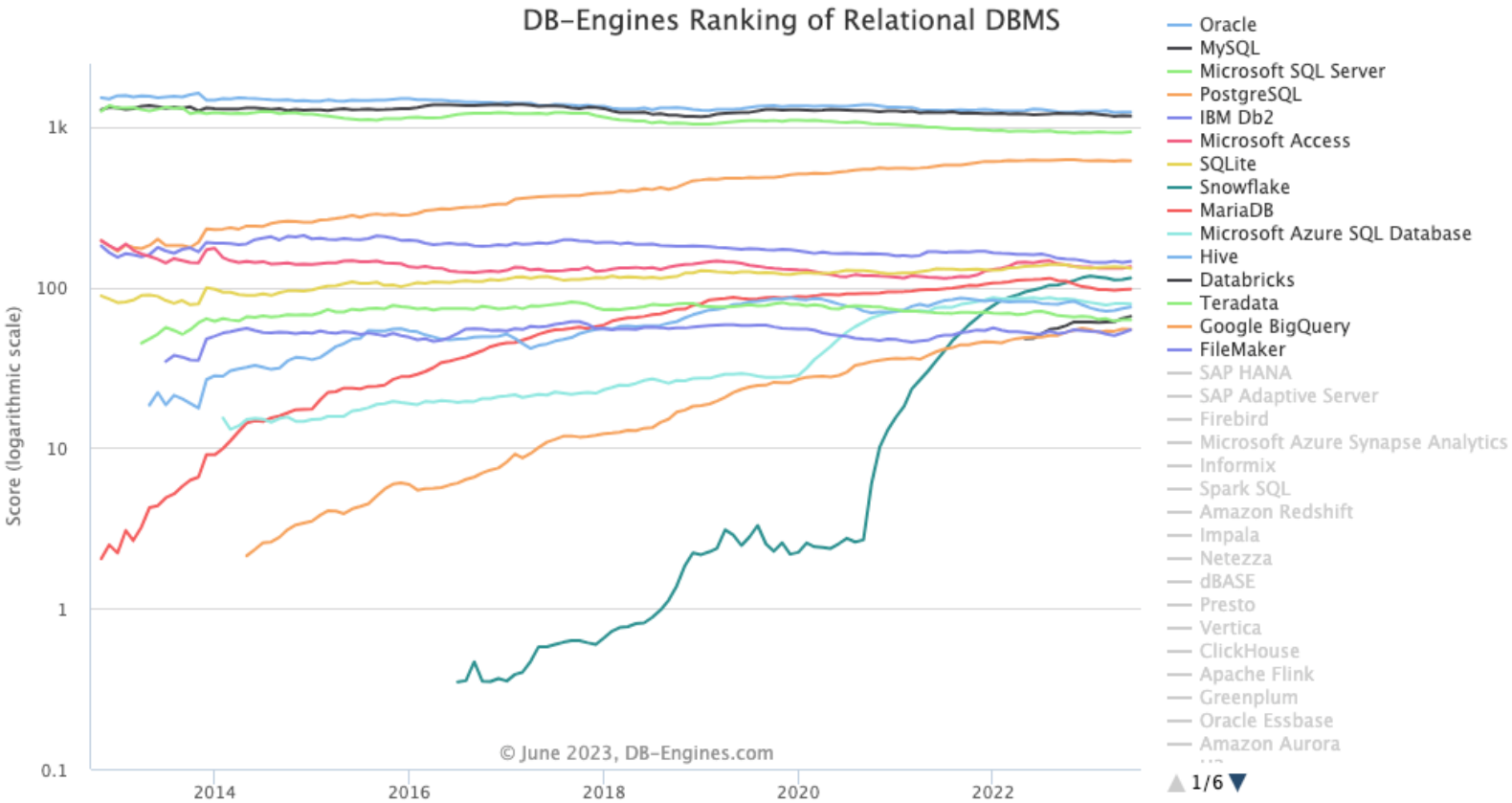
- Generation
- Storage
- Ingestion
- Transformation
- Serving

Src: Fundamentals of Data Engineering (Reis & Housley)

Data Storage

- We usually have the tendency to think thru data as “tabular” with connectivity & relationships in between. SQL Servers (Oracle, MySQL, MS SQL….etc) are one of the most used data stores . SQL Databases or “Relational” Databases have been created around 1970.
- For Reporting, best to leverage “star schemas” where your dimensions are in the middle and the side tables provide the metrics/further details. (Kimball Data Warehouse Books are good reference).
- The SQL query language is based on set theory and is mostly cross platform.

Data Storage



https://www.linkedin.com/pulse/database-engine-ranking-popularity-2023-tamer-khraisha-ph-d-/

Data Storage - NoSQL

1. **NoSQL databases**: NoSQL databases are non-relational databases that can handle unstructured, semi-structured, or differently structured data. They are designed for horizontal scalability and high availability. NoSQL databases can be further classified into several categories:

- Document-oriented databases (e.g., MongoDB, Couchbase)
- Key-value stores (e.g., Redis, Amazon DynamoDB)
- Column-family stores (e.g., Apache Cassandra, HBase)
- Graph databases (e.g., Neo4j, Amazon Neptune)

2. **Time-series databases**: These databases are designed to store and manage time-series data, which consists of data points indexed by time. Time-series databases are optimized for high write and query performance, data retention, and handling large volumes of time-stamped data. Examples include InfluxDB, OpenTSDB, and TimescaleDB.

Data Storage - NoSQL

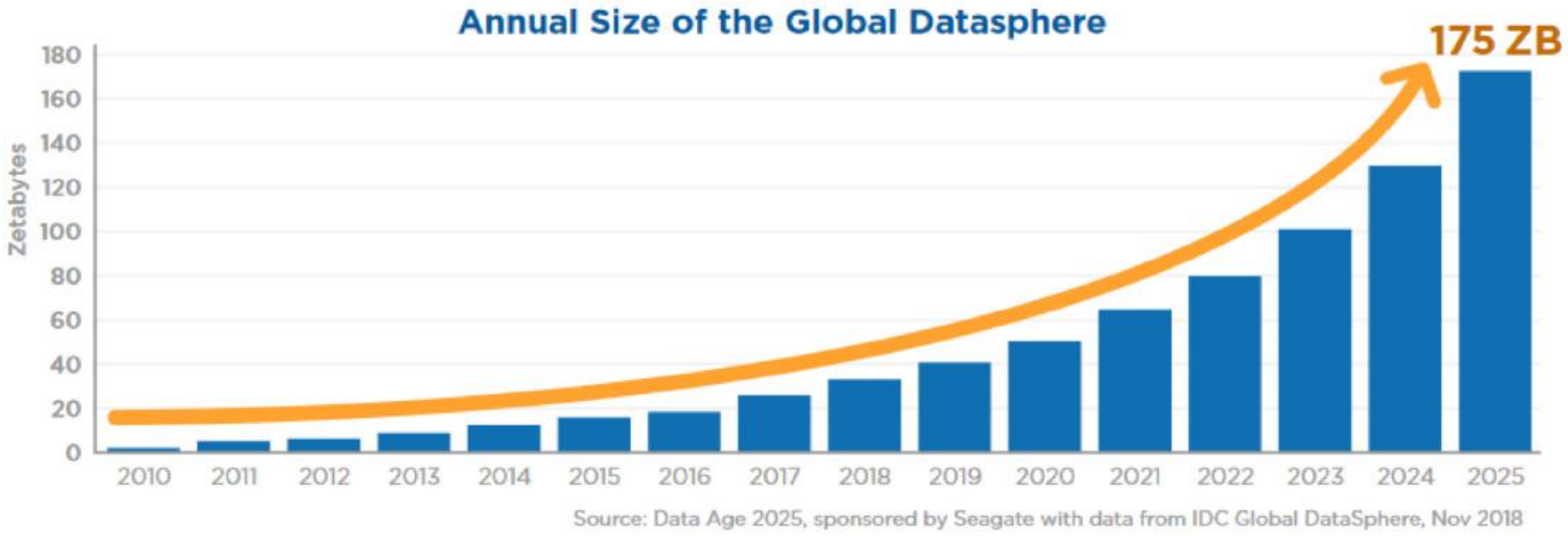
3. **Object storage**: Object storage is a data storage architecture that manages data as objects, rather than files or blocks. It is designed for high scalability, durability, and cost-efficiency, making it suitable for storing and managing large amounts of unstructured data, such as multimedia files, backups, and archives. Examples include Amazon S3, Google Cloud Storage, and Microsoft Azure Blob Storage.

4. **Distributed file systems**: Distributed file systems allow files to be stored across multiple servers or nodes, providing high availability, fault tolerance, and horizontal scalability. These systems are particularly useful for managing large files or datasets that need to be accessed and processed concurrently by multiple users or applications. Examples include Hadoop Distributed FileSystem (HDFS), GlusterFS, and Ceph.

Data Storage - NoSQL

5.**In-memory data stores**: In-memory data stores store data in the main memory (RAM) of a computer or a cluster of computers, providing low-latency access and high throughput for read and write operations. They are often used for caching, real-time data processing, and analytics. Examples include Redis (in-memory key-value store) and Apache Ignite (in-memory data grid).
SAP HANA is one of the latest inventions/stores which is used for both reporting and transactional data store. (Oracle/other companies have other products, like times 10….etc)
6. **File Based**: e.g. Splunk and Elasticsearch are both powerful platforms used for searching, analyzing, and visualizing large volumes of data. Search Processing Language is splunk's language to query the indexed/on the fly indexed data.

Data in the digital world



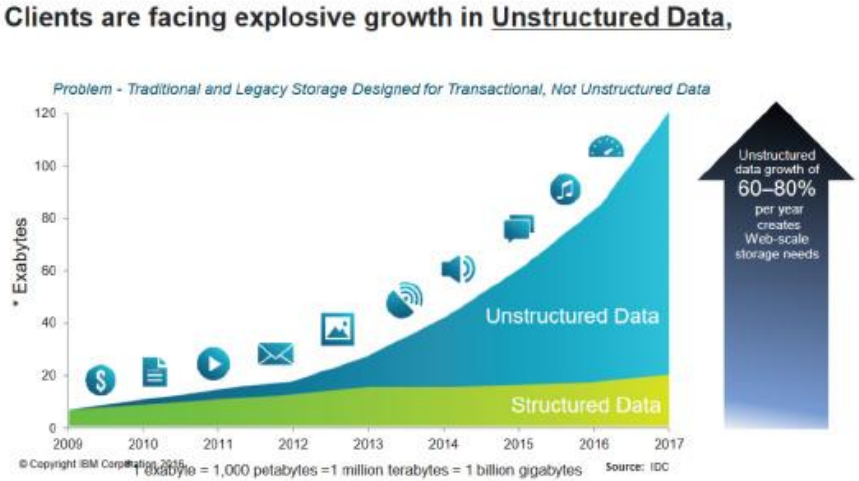
1 Zetta-byte = 1,000,000,000 terabytes (TB)

Src:: https://hexanika.com/big-data/

Data Growth

UNSTRUCTURED DATA:
80% of the worldwide data that will be generated by 2025 will be unstructured because of:

- Heterogeneous Sources: Due to data from multiple sources being pushed into a centralized data repository (Data Lake / Data Warehouse)
- Data Quality: Multiple file formats & types
- Thirdly, Governance & Auditability: Increased data management costs and Lack of business insights & analytics



Src:: https://hexanika.com/big-data/

Big Data Characteristics

The main elements of Big Data are:

Volume - There is a massive amount of data generated every second.

Velocity - The speed at which data is generated, collected, and analyzed

Variety - The different types of data: structured, semi-structured, unstructured

Value - The ability to turn data into useful insights for your business

Veracity - Trustworthiness in terms of quality and accuracy

Src:https://www.simplilearn.com/tutorials/hadoop-tutorial/what-is-hadoop

Hadoop

- Open-source framework for distributed storage and processing of large data sets
- Developed by: Doug Cutting and Mike Cafarella in 2006
- Inspired by: Google's MapReduce and Google File System papers
- Core components: HDFS (storage) and MapReduce (processing)

The Hadoop Ecosystem"

- HDFS (Hadoop Distributed File System)
- MapReduce (Processing paradigm)
- YARN (Yet Another Resource Negotiator)
- Hive (SQL-like queries)

Hadoop

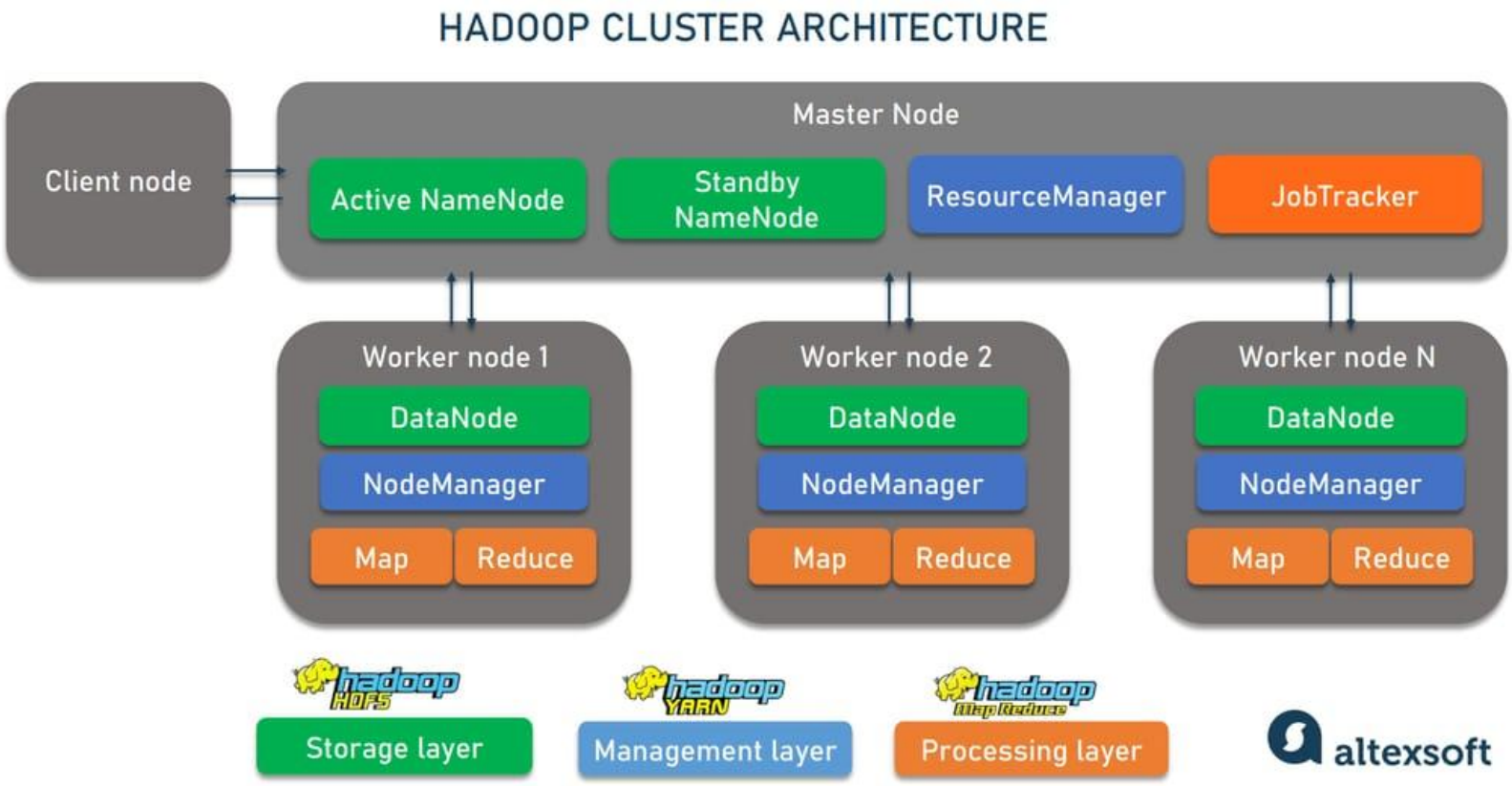
- Data distribution: Large files split into blocks across nodes
- HDFS: Manages data replication and distribution
- MapReduce:
  - Map: Process data in parallel across nodes
  - Shuffle and Sort: Aggregate intermediate results
  - Reduce: Final processing to produce output

Src:https://www.simplilearn.com/tutorials/hadoop-tutorial/what-is-hadoop

Hadoop

- Impacts:
  - Enabled processing of massive datasets
  - Sparked the "Big Data" revolution
  - Influenced development of numerous other big data technologies
- Limitations:
  - Primarily batch-oriented processing
  - Complex programming model
  - Inefficient for iterative algorithms
  - Lack of real-time processing capabilities

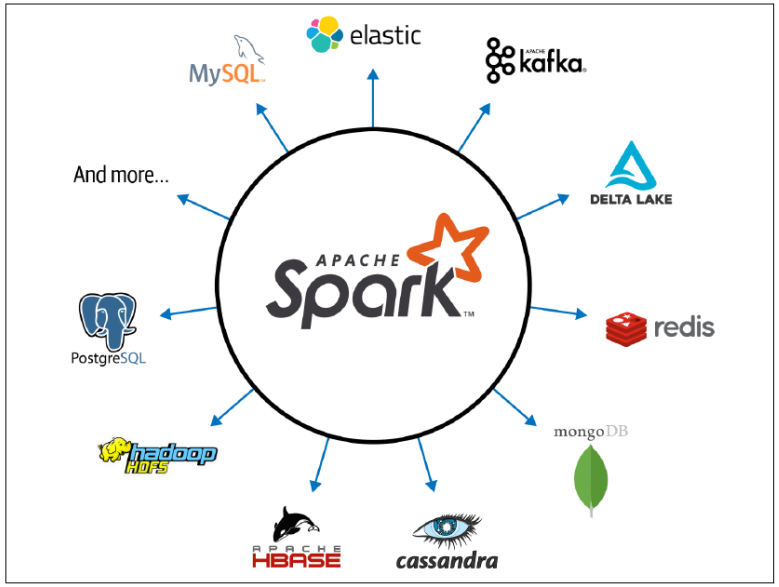
Hadoop



Src:https://www.altexsoft.com/blog/hadoop-pros-cons/

Spark

Apache Spark is a unified analytics engine for large-scale data processing. It provides high-level APIs in Java, Scala, Python, and R, and an optimized engine that supports general execution graphs. Spark was originally developed at UC Berkeley in 2009 and later became an Apache Software Foundation project in 2013.



Src:https://www.simplilearn.com/tutorials/hadoop-tutorial/what-is-hadoop

Spark

Key Features

- Speed: Spark can be up to 100 times faster than Hadoop MapReduce for certain big data tasks, primarily due to its in-memory processing capabilities.
- Ease of Use: Spark offers simple APIs for operating on large datasets, including over 80 high-level operators that make it easy to build parallel apps.
- Generality: Spark supports a wide range of data processing tasks, from simple batch jobs to complex algorithms.
- Runs Everywhere: Spark can run on Hadoop, Apache Mesos, Kubernetes, standalone, or in the cloud. It can access diverse data sources.

Src:https://www.simplilearn.com/tutorials/hadoop-tutorial/what-is-hadoop

Spark - Examples

E-commerce and Retail
Amazon: Uses Spark for personalized product recommendations.
Walmart: Analyzes customer shopping patterns and manages inventory.

Social Media and Networking
LinkedIn: Uses Spark for its news feed algorithm and for analyzing user interactions.
Facebook: Employs Spark for real-time analytics and spam detection.

Finance and Banking
Capital One: Utilizes Spark for credit card fraud detection in real-time.
ING Bank: Uses Spark for customer behavior analysis and risk assessment.

Spark - Examples

Transportation and Logistics
Uber: Uses Spark Streaming for real-time analytics on trip data.
NASA: Analyzes telemetry data from space missions.

Telecommunications
Verizon: Processes network logs for performance optimization and security.
Nokia: Analyzes network data for predictive maintenance.

Media and Entertainment
Netflix: Uses Spark for real-time stream processing and recommendation engines.
Spotify: Analyzes listening habits to create personalized playlists.

Energy Sector
Shell: Analyzes sensor data from oil fields for predictive maintenance.
EDF Energy: Processes smart meter data for energy consumption analysis.
.

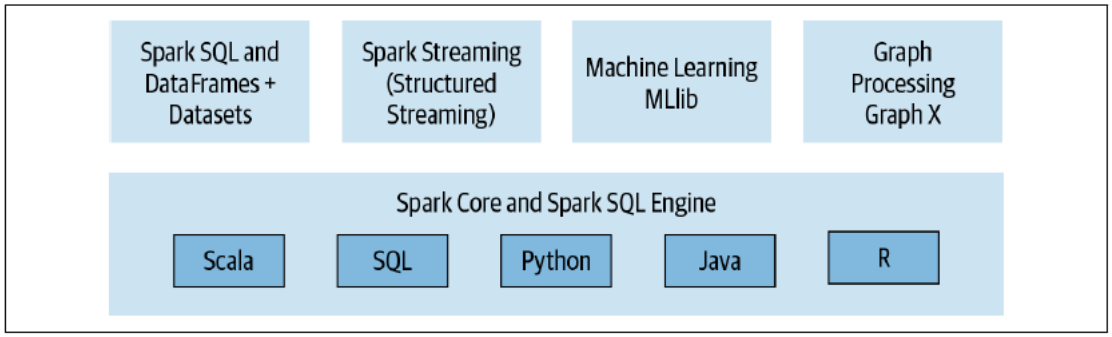
Spark - Examples

Manufacturing
Toyota: Uses Spark for analyzing production line data and improving efficiency.
Siemens: Processes IoT data from industrial equipment for predictive maintenance.

Education
Coursera: Analyzes learner behavior data to improve course recommendations.
edX: Processes large volumes of student interaction data.

Cybersecurity
Symantec: Uses Spark for analyzing security logs and detecting threats.
FireEye: Processes large volumes of network traffic data for threat detection.

Spark



Src: Learning Spark – 2nd edition

Spark

Components of Spark

- Spark Core: The foundation of the entire project, providing distributed task dispatching, scheduling, and basic I/O functionalities.
- **Spark SQL**: Module for working with structured data, allowing querying data via SQL as well as the Apache Hive variant of SQL.

  You read data stored in an RDBMS table or from file formats with structured data (CSV, text, JSON, Avro, ORC, Parquet, etc.) and then construct permanent or temporary tables in Spark. Also, when using Spark's Structured APIs in Java, Python, Scala, or R, you can combine SQL-like queries to query the data just read into a Spark DataFrame

Src:https://www.simplilearn.com/tutorials/hadoop-tutorial/what-is-hadoop

Spark

Spark SQL: read from a JSON file stored on Amazon S3, create a temporary table, and issue a SQL-like query on the results read into memory as a Spark DataFrame:

```
// In Scala
// Read data off Amazon S3 bucket into a Spark DataFrame
spark.read.json("s3://apache_spark/data/committers.json")
.createOrReplaceTempView("committers")
// Issue a SQL query and return the result as a Spark DataFrame
val results = spark.sql("""SELECT name, org, module, release, num_commits
FROM committers WHERE module = 'mllib' AND num_commits > 10
ORDER BY num_commits DESC""")
```

Src:https://www.simplilearn.com/tutorials/hadoop-tutorial/what-is-hadoop

Spark

MLlib: A distributed machine learning framework. library containing common machine learning (ML) algorithms called MLlib.

Streaming: Continuous Streaming model and Structured Streaming APIs, built atop the Spark SQL engine and DataFrame-based APIs. By Spark 2.2, Structured Streaming was generally available, meaning that developers could use it in their production environments.

GraphX: A distributed graph processing framework. GraphX is a library for manipulating graphs (e.g., social network graphs, routes and connection points, or network topology graphs) and performing graph-parallel computations.

Src:https://www.simplilearn.com/tutorials/hadoop-tutorial/what-is-hadoop

Installing Spark

1. Standalone Mode
Simplest way to run Spark in a distributed environment Spark manages its own cluster of worker nodes Ideal for testing and small-scale deployments
Easy to set up and doesn't require additional cluster management software

2. YARN (Yet Another Resource Negotiator) Mode Runs Spark on Hadoop YARN clusters
Two sub-modes:
a) YARN Client Mode: Driver runs on the client machine
b) YARN Cluster Mode: Driver runs on one of the cluster nodes
Beneficial for organizations already using Hadoop Allows for resource sharing with other frameworks like MapReduce

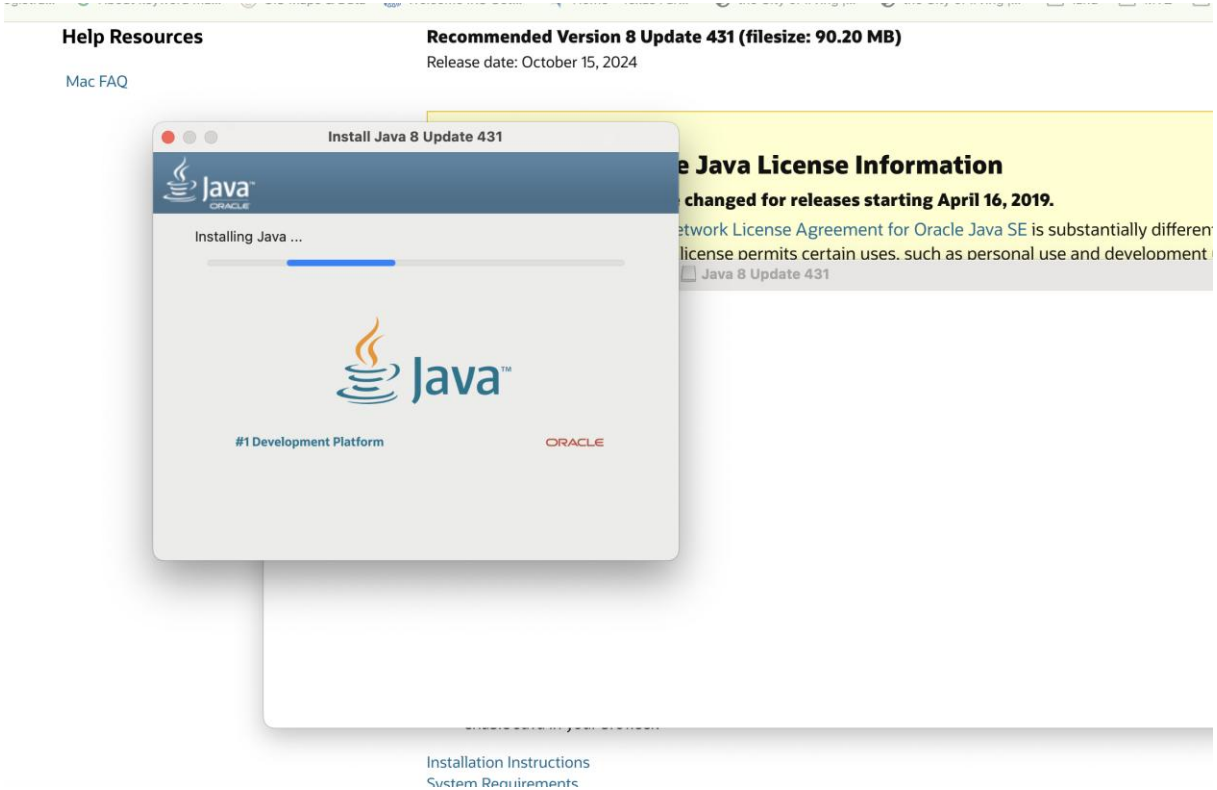
Installing Spark

3. Mesos Mode : Runs Spark on Apache Mesos clusters
Offers fine-grained resource sharing between Spark and other applications
Two modes:
a) Coarse-grained: Allocates fixed resources to each Spark executor
b) Fine-grained: Dynamically adjusts resources for tasks

4. Kubernetes Mode: Runs Spark on Kubernetes clusters Leverages Kubernetes for resource management and scheduling Beneficial for cloud-native applications
Supports dynamic allocation of resources

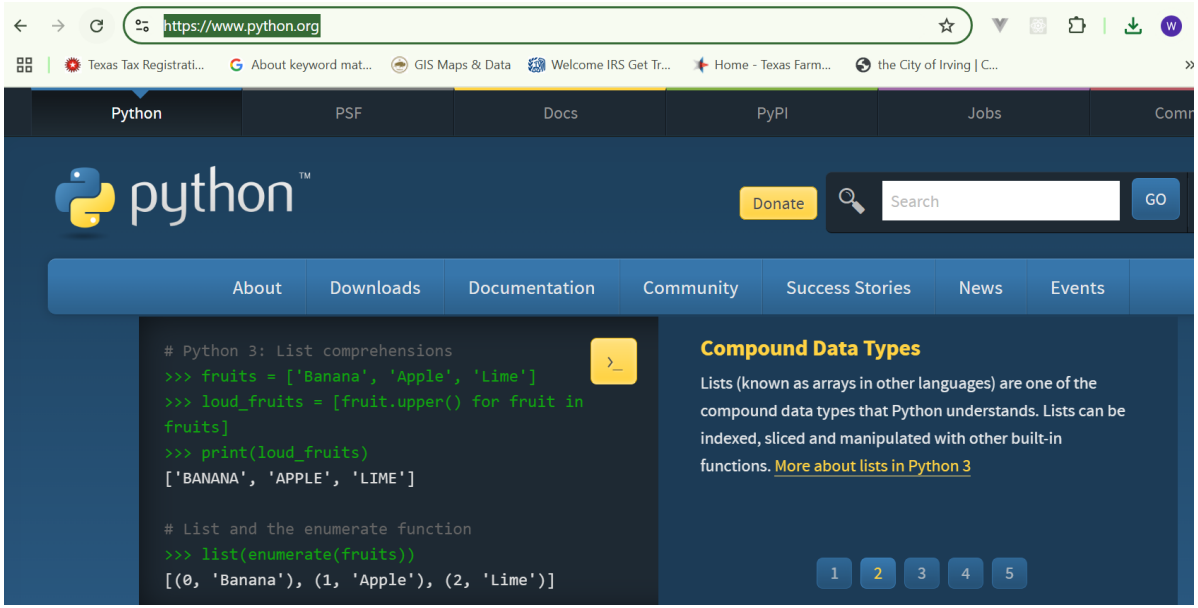
5. Local Mode: Runs Spark on a single machine Uses threads instead of distributed workers Primarily used for development, testing, and learning purposes
Not suitable for processing large datasets

Installing Spark



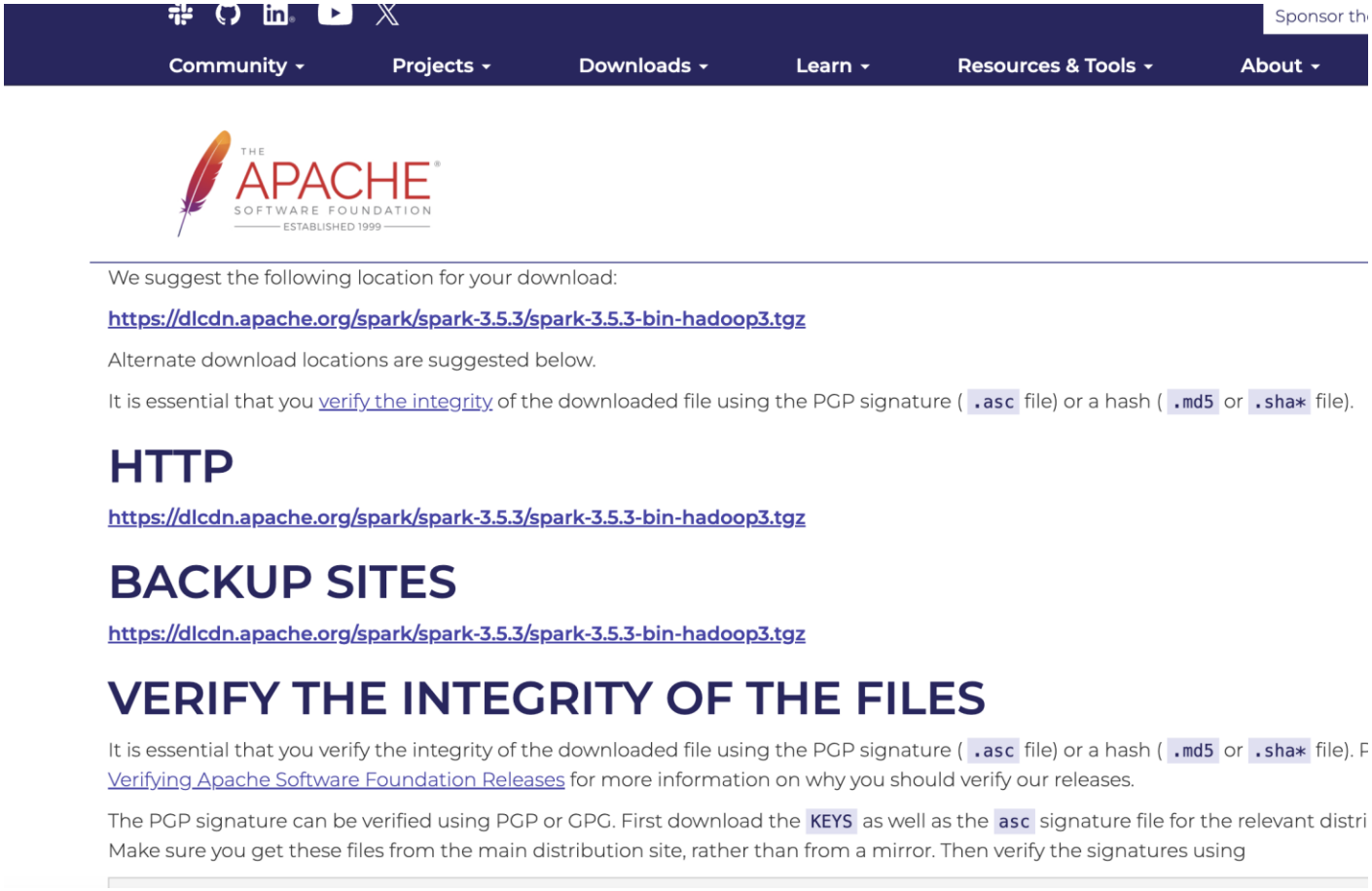
Please note down where you installed Java…usually it is in /usr/libexec/java_home
Spark is not compatible with all Java versions (8 recommended/11).

Installing Spark



Please note down where you installed Python…PATH="/Library/Frameworks/Python.framework/Versions/3.11/bin:${PATH}" export PATH and is updated automatically

Installing Spark



Installing Spark

UnPack the tar file in the directory of choice and noted that down…

```
tar -xf <spark file name that you just downloaded from spark site>
```

Check that the directory has been unpacked (bin subdirectory…etc)

Installing Spark

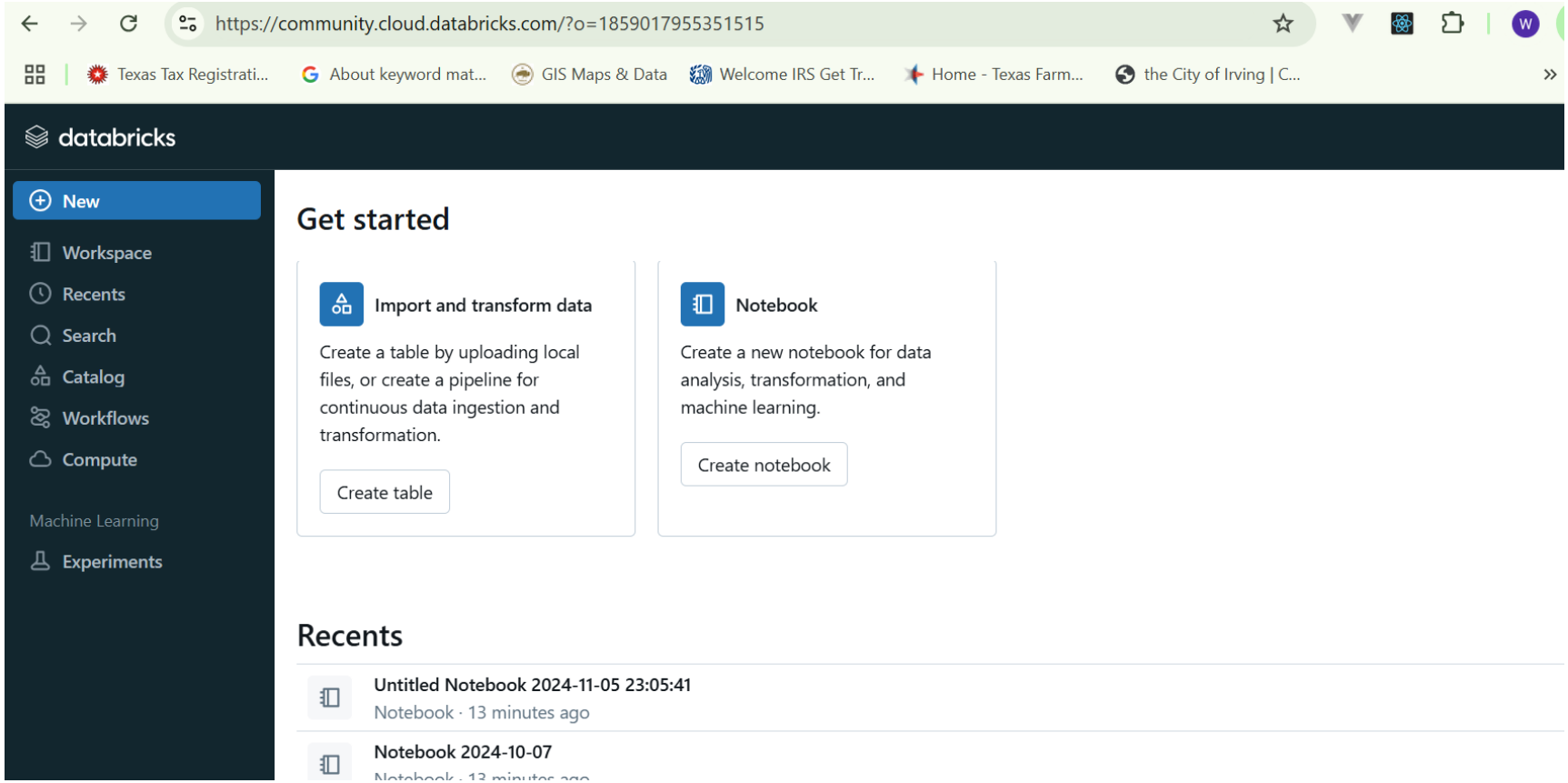
Update your profile (.zprofile) and refresh it…

```
# Setting PATH for Python 3.11
# The original version is saved in .zprofile.pysave
PATH="/Library/Frameworks/Python.framework/Versions/3.11/bin:${PATH}"
export PATH


# Setting JAVA_HOME
export JAVA_HOME=$(/usr/libexec/java_home)
export PATH="$JAVA_HOME/bin:$PATH"


# Setting SPARK_HOME
export SPARK_HOME="/Users/wnhassan/WalidMac/GWMac/spark-3.5.3-bin-hadoop3"
export PATH="$SPARK_HOME/bin:$PATH"


# Setting PYSPARK_PYTHON
export PYSPARK_PYTHON=python
```

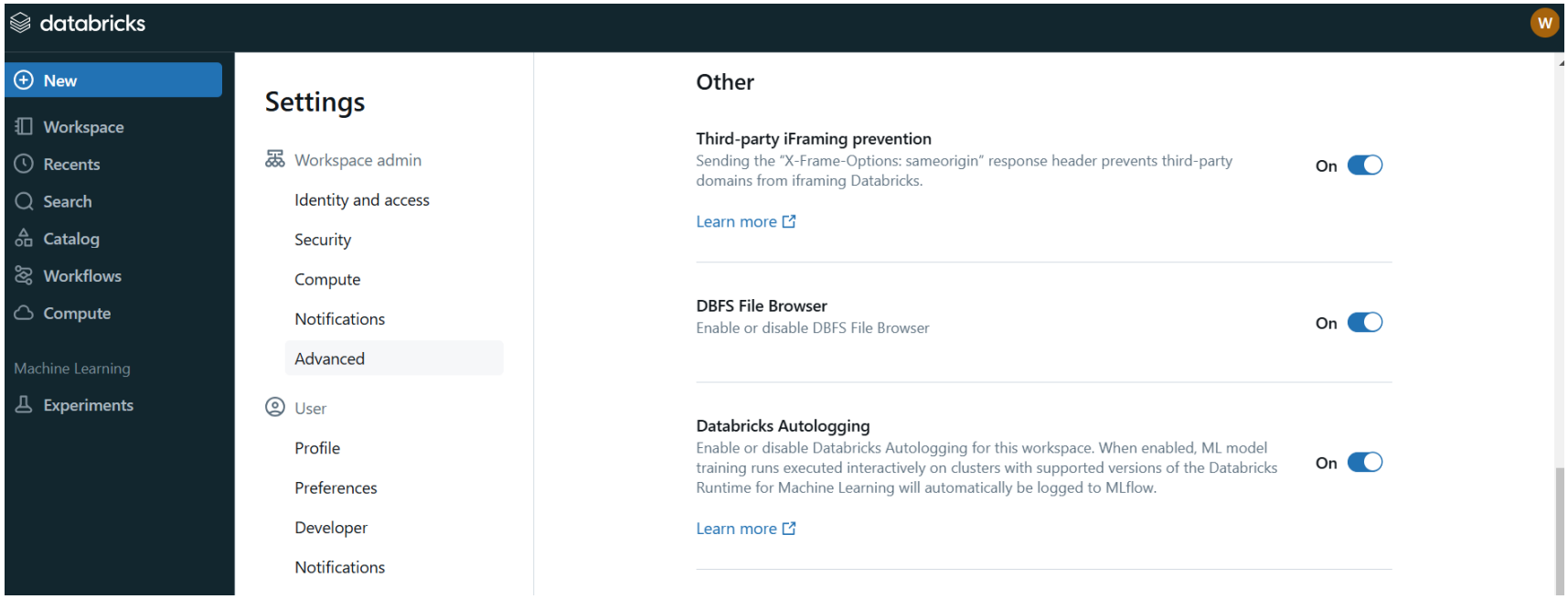
Installing Spark

```
Python 3.11.4 (v3.11.4:d2340ef257, Jun  6 2023, 19:15:51) [Clang 13.0.0 (clang-1300.0.29.30)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
24/11/05 17:53:46 WARN Utils: Your hostname, Walids-MacBook-Pro.local resolves to a loopback address: 127.0.0.1; using 10.0.0.13 i
stead (on interface en0)
24/11/05 17:53:46 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
24/11/05 17:53:47 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes whe
 applicable
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /__ / .__/\_,_/_/ /_/\_\   version 3.5.3
      /_/

Using Python version 3.11.4 (v3.11.4:d2340ef257, Jun  6 2023 19:15:51)
Spark context Web UI available at http://10.0.0.13:4040
Spark context available as 'sc' (master = local[*], app id = local-1730850828533).
SparkSession available as 'spark'.
>>> strings = spark.read.text("../README.md")
>>> strings.show(10, truncate=False)
+-----------------------------------------------------------------------------------------------------------------------+
|value                                                                                                                  |
+-----------------------------------------------------------------------------------------------------------------------+
|# Apache Spark                                                                                                         |
|                                                                                                                       |
|Spark is a unified analytics engine for large-scale data processing. It provides                                       |
|high-level APIs in Scala, Java, Python, and R, and an optimized engine that                                            |
|supports general computation graphs for data analysis. It also supports a                                              |
|rich set of higher-level tools including Spark SQL for SQL and DataFrames,                                             |
|pandas API on Spark for pandas workloads, MLlib for machine learning, GraphX for graph processing,                    |
|and Structured Streaming for stream processing.                                                                        |
|                                                                                                                       |
|<https://spark.apache.org/>                                                                                            |
+-----------------------------------------------------------------------------------------------------------------------+
only showing top 10 rows

>>> strings.count()
125
>>> exit()
wnhassan@Walids-MacBook-Pro bin %
```

Spark in the Cloud

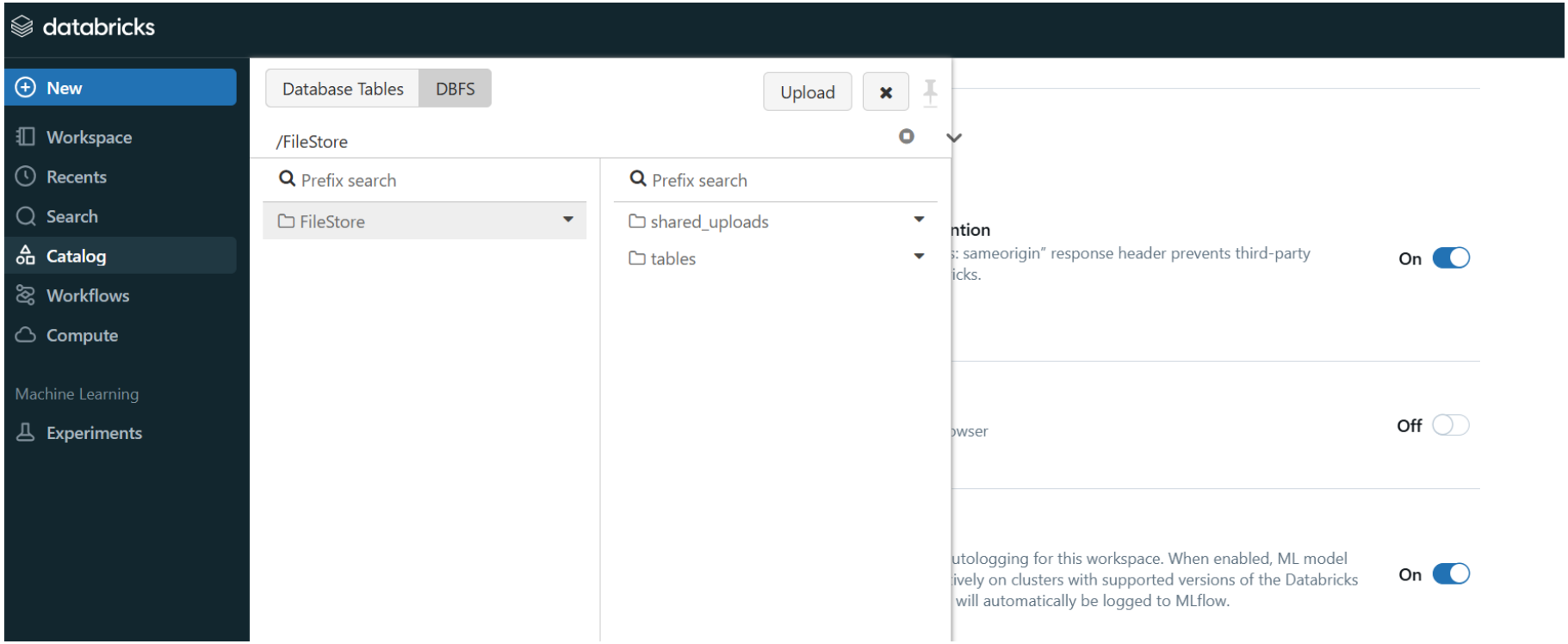


Spark



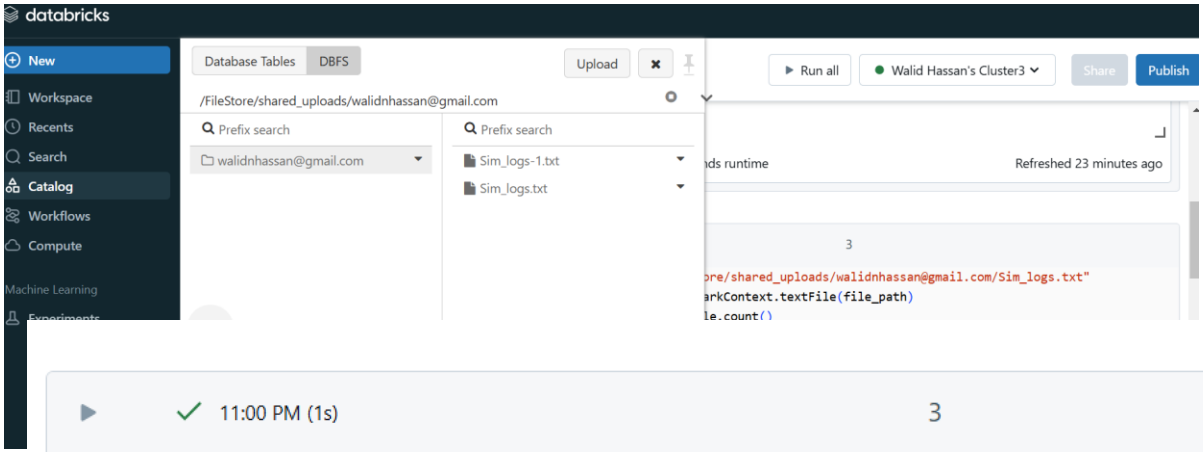
To enable visual data browser, you need to go to the settings and explicitly check that

Spark



The DBFS is now visible…. You can also use python to list the files:
display(dbutils.fs.ls("/FileStore/"))

Spark



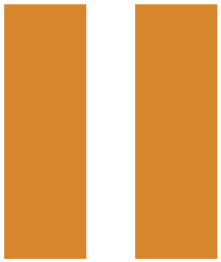
11:00 PM (1s) 3

```
file_path = "/FileStore/shared_uploads/walidnhassan@gmail.com/Sim_logs.txt"
text_file = spark.sparkContext.textFile(file_path)
line_count = text_file.count()
print(f"Number of lines in the file: {line_count}")
```

(1) Spark Jobs

```
Number of lines in the file: 50000
```

You access the uploaded files from your python code…



BREAK

Please come back @

2:37 PM EST

1:37 PM CST

References

In addition to the references in each slide the below are leveraged throughout this course.

Gareth James, D. W. (2023). An Introduction to Statistical Learning: with Applications in Python. Springer.

(Jules S. Damji, July 2020), Learning Spark Lightning Fast Data Analytics , 2020 (2nd Edition).
Databricks Lakehouse Platform Cookbook, Dennis, Dr. Alan L.
Fundamentals of Data Engineering, O'Reilly (J. Reis, M. Housley)